Project 2: Kernel Module and System Call

Due by **September 20, 2024, 11:59 pm**

Summary

Based on the VM, Ubuntu, and Linux kernel that we installed in the previous project, in Project 2, we will create a new module and a new system call to the kernel (6.10) on our Ubuntu VM. This project will be the start of our kernel development journey, and it will help us understand how system calls and kernel modules work in an OS.

Description

Step 1: Implement a new module for your new kernel.

As we will discuss in class, modules are an important means for extending the functionalities of a monolithic OS kernel like Linux. In this step, you will implement a new module to add a function to your kernel. This function should print out the name of our new kernel developer, i.e., *you*, to the kernel log when the module is loaded. Obviously, this is not a typical kernel function, but the new module you will develop here will be extended in the later projects to implement real, useful functions.

Steps:

1. **Implement a basic kernel module**

   You can follow this simple hello world example to create your kernel module source file *my_name.c*. module_init() is called when the module is loaded and module_exit() is called when the module is unloaded. You can place the module code (and the makefile explained next) in a separate folder outside the Linux source code folder.

```c
#include <linux/init.h>
#include <linux/module.h>
#include <linux/kernel.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Linus Torvalds");

static int __init example_init(void) {
    printk(KERN_INFO "Hello, World!\n");
```

Copyright © Ming Zhao, Ph.D.

```
    return 0;
}

static void __exit example_exit(void) {
    printk(KERN_INFO "Goodbye, World!\n");
}

module_init(example_init);
module_exit(example_exit);
```

2. **Implement module parameters**

Use the module_param() macro to pass parameters from command-line to a kernel module. Call this macro at the beginning of your module code.

The following code snippet explains how to use the module_param() macro:

```
/* The module_param() macro takes three arguments:
 - The name of the variable, which also becomes the name of the
   module parameter
 - The type is the data type of the variable
 - The last argument is perm, which denotes permissions. This can be
   left as zero
static int intParameter = 0;
module_param(intParameter, int, 0);

static char *charParameter = “default”;
module_param(charParameter, charp, 0);
```

Your kernel module **MUST** accept **two** parameters: **charParameter=”Fall”** and **intParameter=2024**.

Now change your module code so that when the module is loaded it prints out a message in the following format to the kernel log:

Hello, I am <Your Name>, a student of CSE330 ${charParameter} ${IntParameter}.

Use **printk** to print to the kernel log. It is a powerful tool for kernel debugging. Refer printk documentation for details.

Copyright © Ming Zhao, Ph.D.

3. **Compile the kernel module**

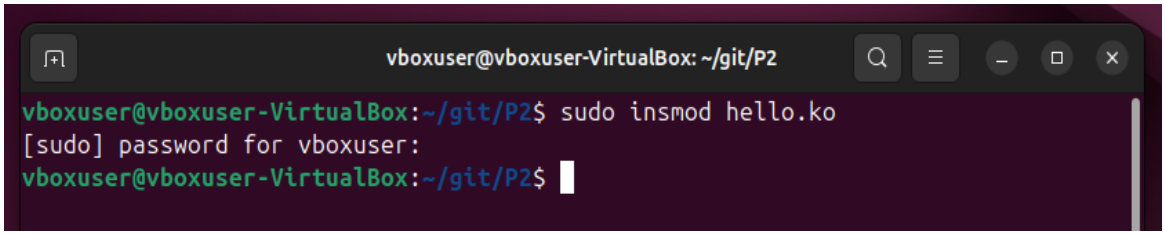
   Create a makefile for compiling the new kernel module.

```
obj-m += my_name.o
all:
	make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules
clean:
	make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

   After creating the makefile, you can use the '*make*' command in the terminal to compile your kernel module.

4. **Load the kernel module**

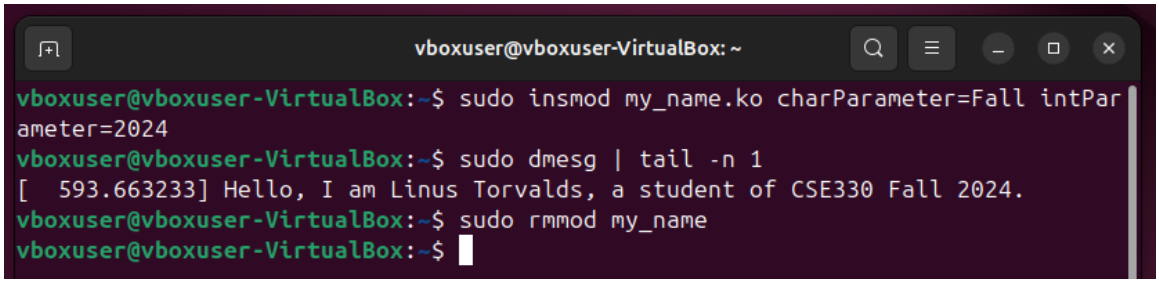
   After a successful build, you will see the hello.ko file generated. Now load the kernel module using `*insmod*` (with *sudo*).



   Check the module output using the *dmesg* command (Kernel logs are stored under /var/log/messages. You can also use the *dmesg* command. After you insert the kernel module, the output can be checked using sudo dmesg | tail -n 1).

   Remove the module using rmmod and it should be successful. You can use lsmod to check if the module is indeed removed.

   The following snapshot shows all the commands used for testing and their outputs:



Congrats! Now you know how to create custom kernel modules.

Copyright © Ming Zhao, Ph.D.

Step 2: Implement a new system call for your new kernel.

As we will also discuss in class, system calls are the main interface for user-space software to interact with the kernel. In this step, you will implement a new system call that simply prints out one message in the kernel log and a user-space program that tests this system call.

Steps:

1. Make a subfolder for your new system call's source code and makefile in your Linux source code folder. *e.g. linux-6.10/my_name*

2. Create the system call source file *my_syscall.c* and define the new system call
   A. **x86**: Name your function `__x64_sys_my_syscall` in your code file, *e.g. linux-6.10/my_name/my_syscall.c*

```c
#include <linux/kernel.h>

asmlinkage long __x64_sys_my_syscall(void) {
    // ...
    return 0;
}
```

   B. **ARM**: Name your function `__arm64_sys_my_syscall` in your code file, *e.g. linux-6.10/my_name/my_syscall.c*

```c
#include <linux/kernel.h>
#include <linux/syscalls.h>

asmlinkage long __arm64_sys_my_syscall(void) {
    // ...
    return 0;
}
```

   Use printk() in your system call implementation to print out "This is the new system call Linus Torvalds implemented." (Replace "Linus Torvalds" with your full name).

3. Create a new makefile in the same folder. Add `obj-y := my_syscall.o` to your new **Makefile**. *e.g. linux-6.10/my_name/Makefile*

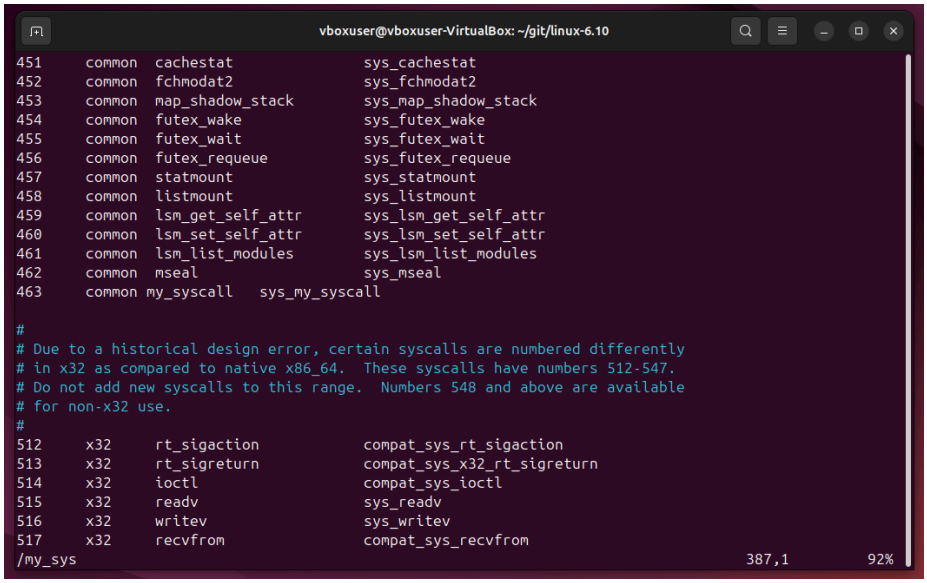
4. Associate the new makefile to the kernel's makefile. This means that you need to modify 'linux-6.10/Makefile'. You need to search for the '`core-y :=`' and add your folder name after it, as shown below.

```
741
742 ifeq ($(KBUILD_EXTMOD),)
743 # Objects we will link into vmlinux / subdirs we need to visit
744 core-y          := my_name/
745 drivers-y       :=
746 libs-y          := lib/
747 endif # KBUILD_EXTMOD
748
```

   Add the new system call to the system call table.
   A. **x86**: File Location: `linux-6.10/arch/x86/entry/syscalls/syscall_64.tbl`

Copyright © Ming Zhao, Ph.D.

B. **ARM**: File Location: linux-6.10/arch/arm/tools/syscall.tbl



Syscall table for x86

5. Add the new system call's prototype in the system call header file. For both x86 and ARM the file path is the same. linux-6.10/include/linux/syscalls.h

```
asmlinkage long sys_mincore(unsigned long start, size_t len,
                            unsigned char __user * vec);
asmlinkage long sys_madvise(unsigned long start, size_t len, int behavior);
asmlinkage long sys_process_madvise(int pidfd, const struct iovec __user *vec,
                        size_t vlen, int behavior, unsigned int flags);
asmlinkage long sys_process_mrelease(int pidfd, unsigned int flags);
asmlinkage long sys_remap_file_pages(unsigned long start, unsigned long size,
                        unsigned long prot, unsigned long pgoff,
                        unsigned long flags);
asmlinkage long sys_mseal(unsigned long start, size_t len, unsigned long flags);
asmlinkage long sys_my_syscall(void);
asmlinkage long sys_mbind(unsigned long start, unsigned long len,
                            unsigned long mode,
                            const unsigned long __user *nmask,
                            unsigned long maxnode,
                            unsigned flags);
asmlinkage long sys_get_mempolicy(int __user *policy,
                            unsigned long __user *nmask,
                            unsigned long maxnode,
                            unsigned long addr, unsigned long flags);
```

6. Update the system call numbers, which embeds the pointer to the syscall in the table
   A. File: linux-6.10/include/uapi/asm-generic/unistd.h for both x86 and ARM

Copyright © Ming Zhao, Ph.D.

```
#define __NR_lsm_list_modules 461
__SYSCALL(__NR_lsm_list_modules, sys_lsm_list_modules)

#define __NR_mseal 462
__SYSCALL(__NR_mseal, sys_mseal)

#define __NR_sys_my_syscall 463
__SYSCALL(__NR_sys_my_syscall, sys_my_syscall)

#undef __NR_syscalls
#define __NR_syscalls 464
```

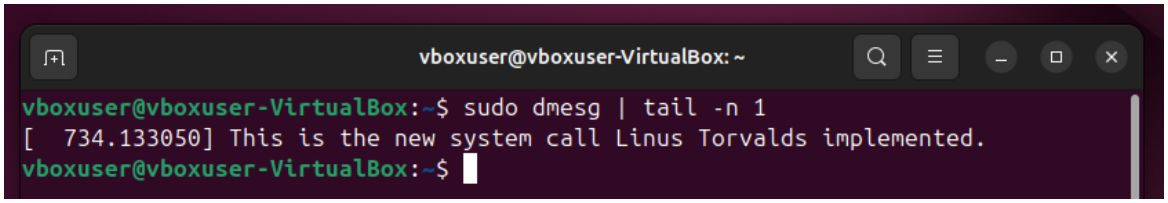
B. File: linux-6.10/arch/arm64/include/asm/unistd.h for ARM only

```
#define __ARM_NR_COMPAT_BASE            0x0f0000
#define __ARM_NR_compat_cacheflush      (__ARM_NR_COMPAT_BASE + 2)
#define __ARM_NR_compat_set_tls         (__ARM_NR_COMPAT_BASE + 5)
#define __ARM_NR_COMPAT_END             (__ARM_NR_COMPAT_BASE + 0x800)

#define __NR_compat_syscalls            464 //incremented from 463 -> 464
#endif

#define __ARCH_WANT_SYS_CLONE
```

7. Recompile your kernel following the steps we did in Project 1 (this will take a while):
   A. make -j4
   B. sudo make modules_install -j4
   C. sudo make install -j4
   D. sudo update-grub
8. Reboot your VM into the new kernel.
9. Write a userspace program *syscall_in_userspace_test.c* for invoking your new system call following this template.
10. Compile the userspace program and run it.

```
gcc -o syscall_in_userspace syscall_in_userspace_test.c
./syscall_in_userspace
```

After you invoke the system call, the expected output from dmesg | tail should be like:



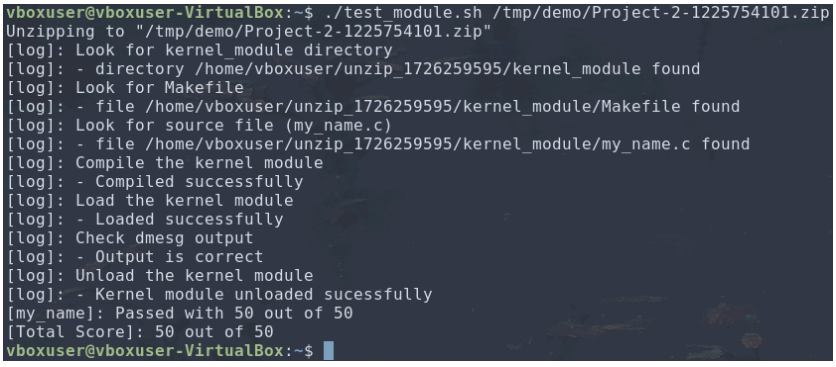
Copyright © Ming Zhao, Ph.D.

Tips:

- It is highly recommended to take VM snapshots after you complete each major step in order to keep a working backup.
- Periodically delete outdated kernel versions if you are running out of space
  - **dpkg --list 'linux-image-*'**
  - Find all the kernels that are lower than your current kernel. When you know which kernel to remove, continue below to remove it. (Do not remove the original kernel that comes with Ubuntu so you have a working backup.)
    Run the commands below to remove the kernel you selected.
    sudo apt-get purge linux-image-x.x.x-x-generic
  - **sudo update-grub2**
  - Reboot your VM
- Also delete the outdated system-level logs in order to avoid low disk space errors/warnings. This can be accomplished by typing sudo dmesg -C into your terminal.

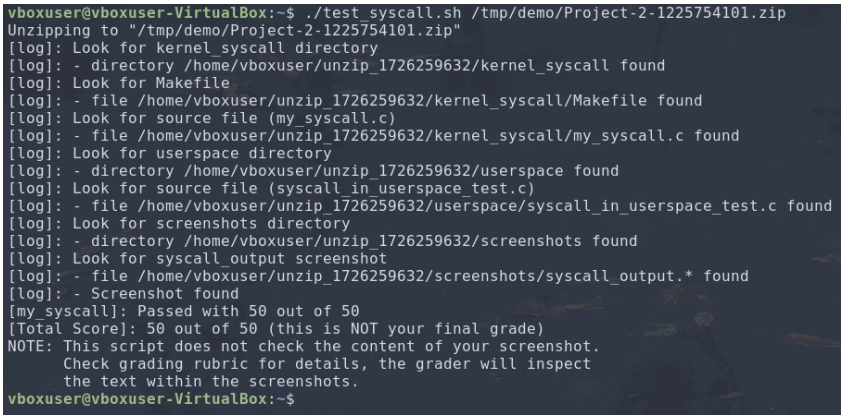
Testing

- In order to test your project we have provided you with the grading scripts. These are available on a GitHub repository. If you have not used git before you can use this link to familiarize yourself.
- In order to clone the Github repository follow the below steps
  - git clone https://github.com/visa-lab/CSE330-OS.git
  - cd CSE330-OS/
  - git checkout project-2
- Use test_module.sh to check the correctness of your kernel module
  - The script will ensure your kernel module source files adhere to the directory structure, compile your module, load it, and unload it.
  - The script will print a detailed log after testing each part of the rubrics and point out reasons if grade points were deducted.
  - Here is a screenshot of a test that passess all checks:



Copyright © Ming Zhao, Ph.D.

- Use test_syscall.sh to test your syscall submission
    - The script will ensure your screenshot adheres to the directory structure and validate the screenshot is present for testing. Graders will check the correctness of the screenshot separately.
    - Here is a screenshot of a test that passess all checks:



Note:

We will check your code in addition to running the test script, so passing the testing script does not guarantee getting all the grade points. But if your submission fails to pass the above scripts entirely, you will receive **zero** grade points.

Submission Requirements & Guidelines

Project 2 is due by **September 20, 2024**. Submit your work following the below guidelines:

- Submit the following in a zip file to Canvas:
    - Source code:
        - kernel module - Make a directory named '***kernel_module***', and submit your code file named '*my_name.c*' and your *Makefile*.
        - system call - Make a directory named '***kernel_syscall***', and submit your code file named '*my_syscall.c*' and your *Makefile*.
        - user-space test program - Make a directory named **'*userspace*'**, and submit your code file named '*syscall_in_userspace_test.c*'.
    - Screenshot of the outputs:
        - The screenshot should include the output from Step 2 Implementing a new system call. It should show the invocation of your userspace program and the expected output in dmesg:
          "This is the new system call <YOUR NAME> implemented."
        - You can use VirtualBox's screenshot function to take a screenshot (to take a screenshot in VirtualBox, go to View-> Take Screenshot).
        - For UTM You can use the macOS keyboard shortcut to take a screenshot (Command + Shift + 3).

Copyright © Ming Zhao, Ph.D.

        - You **MUST** make a directory named **'screenshots'** and **MUST** save one screenshot with the following name: *syscall_output.png*
    - Project-1 resubmission:
        - This is only applicable for students who lost points in Project-1 and want to take this opportunity for regrade.
        - Make a directory '***Project1***'
        - There **MUST** be two screenshots with the following names:
            - *lsb_release.png*
            - *uname.png*
- You **MUST** name the zip file following the naming convention "Project-2-YourASUID>.zip", e.g., Project-2-1225754101.zip
    - You should use the following command to create your zip file:
      `zip -r Project-2-1225754101.zip kernel_module/ kernel_syscall/ screenshots/ userspace/`

      Note: Add 'Project1/' in the above command if you are resubmitting Project1:
      `zip -r Project-2-1225754101.zip kernel_module/ kernel_syscall/ screenshots/ userspace/ Project1/`
    - Note that the extension automatically added by Canvas to your zip file's name is okay.
- Do not submit any other source code
- Do not submit any binary
- If your submissions do not adhere to submission guidelines and naming conventions you will receive **zero** grade points

Grading Rubrics

Criteria	Test	Pts
Kernel Module (my_name) 1. Kernel module should load without errors 2. Expected output should appear in dmesg with student's name and the provided module parameters	`./test_my_name.sh <path to directory containing my_name.c and Makefile>` Check for the student's name and correctness of both module parameters. Expected output: **Hello, I am Linus Torvalds, a student of CSE330 Fall 2024.**	Module load successfully (20) Expected output matches (20) Module unloads successfully (10)

Copyright © Ming Zhao, Ph.D.

3. Kernel module should unload without errors		
System Call (my_syscall) The expected output should appear in dmesg with the student's name	gcc -o syscall_in_userspace_test syscall_in_userspace_test.c ./syscall_in_userspace_test dmesg Check for the student's name. Expected output: **This is the new system call Linus Torvalds implemented.**	50
Submission Content Zip file should contain only the required screenshots and source code.	./test_zip_contents.sh <path/to/zip/file.zip> Checking Directory: kernel_module - Directory Found: kernel_module - File Found: kernel_module/my_name.c - File Found: kernel_module/Makefile Checking Directory: kernel_syscall - Directory Found: kernel_syscall - File Found: kernel_syscall/my_syscall.c - File Found: kernel_syscall/Makefile Checking Directory: userspace - Directory Found: userspace - File Found: userspace/syscall_in_userspace_test.c Checking Directory: screenshots - Directory Found: screenshots - File Found: screenshots/syscall_output.png Summary ---------- --	-5 pts for unwanted files

Copyright © Ming Zhao, Ph.D.

	Got 4 out of 4 expected directories Got 6 out of 6 expected files	

Policies

- Late submissions will **absolutely not** be graded (unless you have verifiable proof of emergency). It is much better to submit partial work on time and get partial credit for your work than to submit late for no credit.
- Every student needs to **work independently** on this exercise. We encourage high-level discussions among students to help each other understand the concepts and principles. However, a code-level discussion is prohibited and plagiarism will directly lead to failure of this course. We will use anti-plagiarism tools to detect violations of this policy.
- **The use of generative AI tools is not allowed** to complete any portion of the assignment.

Copyright © Ming Zhao, Ph.D.